UNIVERSIDAD DEL TOLIMA

Elementos de Programación Orientada a Objetos

Sistema de gestión para librería Lumina: modelo orientado a objetos completo y prototipo en consola

Fase 2 — Diagrama de clases refinado v1.0, interfaces, relaciones y prototipo ejecutable

Valery Morales

Camilo Romero

Miguel Quiroga

Ibagué, 2026

Tabla de contenido

Contents

Tabla de contenido 2

Introducción 3

1. Revisión del trabajo anterior (Fase 1) 3

2. Diagrama de clases refinado v1.0 5

3. Diccionario de clases 7

4. Interfaces 11

5. Tabla de relaciones 12

6. Modelo del mundo y modelo de la interfaz 13

Modelo del mundo 13

Modelo de la interfaz 14

7. Prototipo mínimo ejecutable en consola 14

8. Verificación de cumplimiento 15

Referencias 16

Introducción

En la Fase 1 del proyecto, el CIPAS identificó como dominio el sistema de gestión de una librería y elaboró un diagrama gráfico con las clases base, definiendo sus atributos y métodos iniciales. Ese diagrama recibió el visto bueno. En esta Fase 2 se da un paso más: se refina el modelo con visibilidades, tipos de dato y firmas completas, se incorporan relaciones con multiplicidad, se agregan interfaces con sus realizaciones y se comprueba el modelo mediante un prototipo mínimo ejecutable en consola. Este documento presenta esos cuatro productos esperados: diagrama refinado v1.0, tabla de relaciones, diccionario de clases y prototipo.

El sistema modelado, librería Lumina, gestiona inventario (libros, útiles y revistas), punto de venta con factura, clientes con fidelización por niveles, búsqueda, recomendaciones, reportes y alertas de stock, con persistencia temporal en listas en memoria.

1. Revisión del trabajo anterior (Fase 1)

Se recuperó el diagrama gráfico de clases base y se verificó que cada clase tuviera nombre, atributos y métodos iniciales. La verificación fue satisfactoria: Persona, Cliente, Producto (con Libro, UtilEscolar y Revista), Venta, DetalleVenta, NivelCliente y Categoria estaban definidas con identidad y operaciones básicas. Sobre esa base se construyó el refinamiento de la Fase 2.

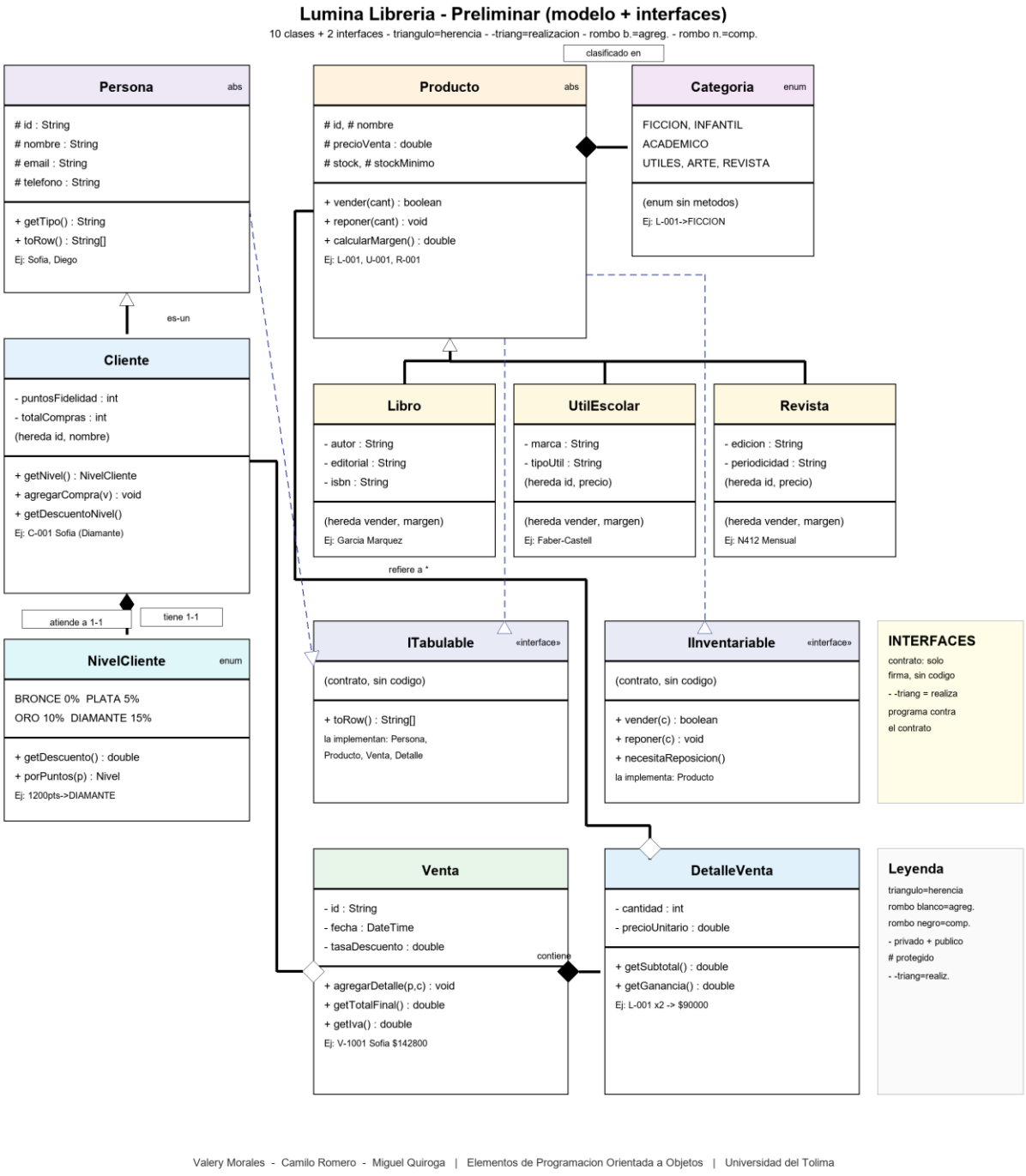


Figura 2

Diagrama de la Fase 1 (clases base verificadas).

Nota. Diagrama preliminar aprobado. Fuente: elaboración propia (2026).

2. Diagrama de clases refinado v1.0

Novedades de la v1.0: La Figura 1 presenta el modelo completo: 10 clases del dominio, 2 enumeraciones, 3 interfaces, además de las clases de persistencia (Database), servicios (ProductoService, VentaService, RecomendacionService, ReporteService) y consola (ConsoleUI, AppMenu). Respecto a la Fase 1 se agregaron visibilidades (+ público, - privado, # protegido), tipos de dato, firmas completas, multiplicidades, las interfaces ITabulable, IInventariable e IPoliticaDescuento con sus realizaciones, y las dependencias entre capas.

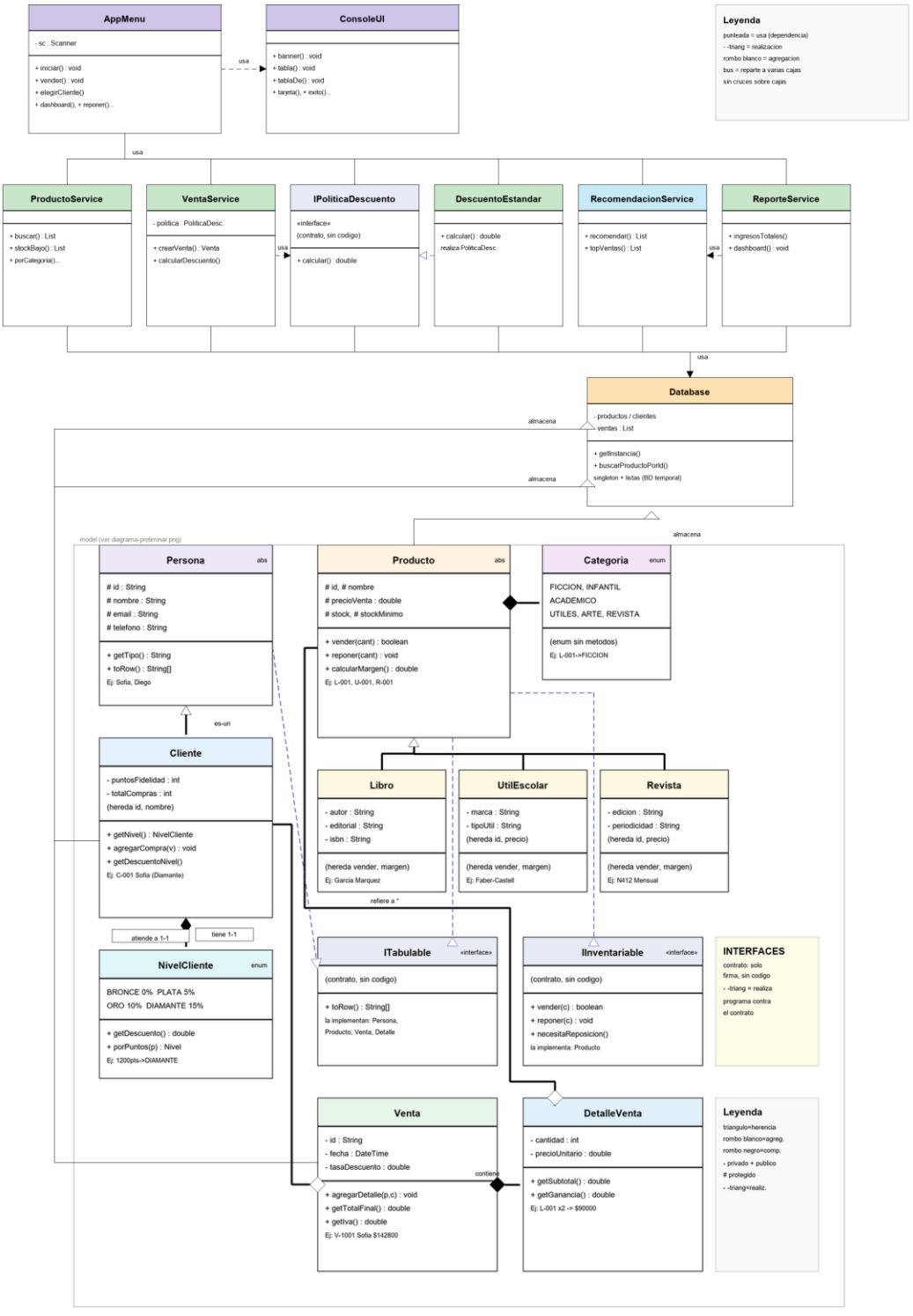
Figura 1

Diagrama de clases refinado v1.0 con atributos, operaciones, interfaces y relaciones.

Nota. Modelo completo del dominio más capas de persistencia, servicios y consola. Fuente: elaboración propia (2026).

Lumina Libreria - Diagrama Completo (21 clases + 3 interfaces)

ui / service / repository / model - punteada=dependencia(usa) - -triang=realizacion



3. Diccionario de clases

Cada clase indica su responsabilidad, sus atributos con visibilidad y tipo, y sus operaciones con firma (parámetros y retorno), conforme a lo pedido en la definición detallada de clases.

Tabla 1

Diccionario de clases: responsabilidad, atributos y operaciones.

Clase	**Responsabilidad**	**Atributos**	**Operaciones**
Persona (abstracta)	Identidad común de las personas del sistema.	#id: String #nombre: String #email: String #telefono: String	+getTipo(): String (abstracto) +toRow(): String[] (abstracto)
Cliente : Persona	Cliente que compra y acumula fidelización.	-puntosFidelidad: int -totalCompras: int	+getNivel(): NivelCliente +getDescuentoNivel(): double +agregarCompra(valor: double): void
NivelCliente (enum)	Niveles cerrados con su descuento.	BRONCE (0 %), PLATA (5 %), ORO (10 %), DIAMANTE (15 %)	+getDescuento(): double +porPuntos(p: int): NivelCliente
Categoria (enum)	Clasificación cerrada del inventario.	LIBRO_FICCION, LIBRO_INFANTIL, LIBRO_ACADEMICO, UTILES, ARTE, REVISTA	+getEtiqueta(): String

Producto (abstracta)	Base de todo lo vendible; stock con reglas.	#id: String #nombre: String #categoria: Categoria #precioCompra: double #precioVenta: double #stock: int #stockMinimo: int	+calcularMargen(): double +necesitaReposicion(): boolean +vender(cantidad: int): boolean +reponer(cantidad: int): void +calcularPrecioFinal(): double (abstracto) +getDetalleEspecifico(): String (abstracto)
Libro : Producto	Libro con identidad editorial.	-autor: String -editorial: String -isbn: String -numPaginas: int	+calcularPrecioFinal(): double +getDetalleEspecifico(): String
UtilEscolar : Producto	Útiles y arte.	-marca: String -tipoUtil: String	+calcularPrecioFinal(): double +getDetalleEspecifico(): String
Revista : Producto	Publicación periódica.	-edicion: String -periodicidad: String	+calcularPrecioFinal(): double +getDetalleEspecifico(): String
DetalleVenta	Línea de factura (producto × cantidad).	-producto: Producto -cantidad: int	+getSubtotal(): double +getGanancia(): double

		-precioUnitario: double	
Venta	Factura con cliente, detalles, descuento e IVA.	-id: String -cliente: Cliente -fecha: LocalDateTime -detalles: List -tasaDescuento: double	+agregarDetalle(prod: Producto, cant: int): void +getSubtotal(): double +getValorDescuento(): double +getIva(): double +getTotalFinal(): double +getGananciaTotal(): double
ITabulable (interfaz)	Contrato de presentación en tablas de consola.	— (sin estado)	+toRow(): String[]
IInventariable (interfaz)	Contrato de inventario con reglas.	— (sin estado)	+getStock(): int +vender(cantidad: int): boolean +reponer(cantidad: int): void +necesitaReposicion(): boolean
Database (singleton)	Base de datos temporal: 3 listas + datos semilla.	-productos: List -clientes: List -ventas: List	+getInstancia(): Database +buscarProductoPorId(id: String): Optional +buscarClientePorId(id: String): Optional

ProductoService	Lógica de inventario y búsqueda.	— (usa Database)	+buscar(query: String): List +porCategoria(cat: Categoria): List +stockBajo(): List +masRentables(top: int): List
VentaService	Cobro con descuentos e IVA; delega en la política.	-politica: IPoliticaDescuento	+VentaService(politica: IPoliticaDescuento) +calcularDescuento(c: Cliente, sub: double, unds: int): double +crearVenta(cliente: Cliente, carrito: List): Venta
IPoliticaDescuento (interfaz)	Contrato Strategy de descuento.	— (sin estado)	+calcular(cliente: Cliente, subtotal: double, unidades: int): double
DescuentoEstandar	Política estándar: nivel + volumen + monto, tope 25 %.	—	+calcular(cliente: Cliente, subtotal: double, unidades: int): double
RecomendacionService	Sugerencias por categoría, autor y popularidad.	— (usa Database)	+recomendar(base: Producto, max: int): List +topVentas(top: int): List +ventasPorProducto(): Map
ReporteService	Dashboard e	— (usa Database)	+ingresosTotales(): double

	indicadores.		+gananciaTotal(): double +ticketPromedio(): double +mejorCliente(): Optional +ventasPorCategoria(): Map
ConsoleUI	Kit visual: colores ANSI y tablas Unicode.	constantes de color	+banner(): void +tabla(headers: String[], filas: List): void +tablaDe(headers: String[], items: List): void +tarjeta(...): void
AppMenu	Flujo del menú 0–10 e interacción con el usuario.	-sc: Scanner	+iniciar(): void +vender(): void +elegirCliente(): Cliente +dashboard(): void +imprimirFactura(v: Venta): void

4. Interfaces

Comportamientos: Se identificaron tres comportamientos comunes definidos como interfaz (convención de nombres con prefijo I):

- **ITabulable — presentación en tablas.** La implementan Persona, Producto, Venta y DetalleVenta. Permite que ConsoleUI.tablaDe() dibuje cualquier lista tabulable sin conocer la clase concreta.

- **IInventariable — inventario con reglas.** La implementa Producto. Garantiza que el stock solo cambia vía vender()/reponer() y que se avisa con necesitaReposicion().
- **IPoliticaDescuento (patrón Strategy) + DescuentoEstandar:** al crear el servicio se le pasa el descuento como parámetro del constructor (new VentaService(new DescuentoEstandar())) y el constructor lo deja guardado; después los métodos solo usan ese objeto ya existente, sin crearlo de nuevo. Así la regla se cambia o se prueba sin tocar el servicio.

5. Tabla de relaciones

Cada relación indica su tipo, su multiplicidad y su justificación.

Tabla 2

Relaciones: tipo, multiplicidad y justificación.

Relación	Tipo	Multiplicidad	Justificación
Persona – Cliente	Herencia	1 – 1	Cliente es-una Persona: reusa identidad y se trata polimórficamente.
Producto – Libro, UtilEscolar, Revista	Herencia (c/u)	1 – 1	Cada una es-un Producto con datos propios (autor, marca, edición).
ITabulable – Persona, Producto, Venta, DetalleVenta	Realización	1 – 1	Las cuatro cumplen el contrato toRow() para dibujarse en tablas.
IInventariable – Producto	Realización	1 – 1	Producto cumple el contrato de stock con reglas.
IPoliticaDescuento – DescuentoEstandar	Realización	1 – 1	La política estándar cumple el contrato Strategy de descuento.
VentaService →	Dependencia	1 – 1	El servicio no fabrica el descuento: le

IPoliticaDescuento	(uso)		llega como parámetro del constructor y sus métodos usan ese objeto ya existente (inyección, DIP).
Cliente – NivelCliente	Asociación (tiene)	1 – 1	El nivel se calcula desde los puntos; no se almacena separado.
Producto – Categoria	Asociación (clasificado en)	* – 1	Muchos productos pertenecen a una categoría del conjunto cerrado.
Venta – Cliente	Agregación (atiende a)	* – 1	La venta referencia al cliente, que existe sin ella.
Venta – DetalleVenta	Composición (contiene)	1 – 1..*	El detalle nace y muere con su venta; toda venta tiene ≥1 detalle.
DetalleVenta – Producto	Agregación (refiere a)	* – 1	El detalle apunta al inventario sin clonarlo ni poseerlo.
Database – Producto, Cliente, Venta	Agregación (almacena)	1 – *	La BD temporal contiene las listas; al cerrar, los datos se pierden.
AppMenu – servicios y ConsoleUI	Dependencia (usa)	1 – 1	El menú orquesta: delega la lógica y lo visual sin contenerlos.

6. Modelo del mundo y modelo de la interfaz

Modelo del mundo

Se modelan las entidades de una librería de barrio: el mostrador con su inventario físico (libros con autor y editorial, útiles con marca, revistas con edición), los clientes del cuaderno de fidelización (con niveles según sus compras), y la factura manual (cabecera con cliente y fecha,

líneas de producto × cantidad, descuento e impuesto). Las listas en memoria hacen las veces de los estantes y del cuaderno mientras el programa corre.

Modelo de la interfaz

Cada entidad se refleja en la consola: el inventario y los clientes en tablas Unicode con colores y alertas de stock; la venta en un carrito numerado (se elige por número o ID, con 0 para crear en memoria) que termina en factura estilizada con subtotal, descuento, IVA y total; los reportes en un dashboard con indicadores y barras ASCII; y todo el flujo en un menú de opciones 0–10 (incluye exportar las ventas a CSV para Excel).

7. Prototipo mínimo ejecutable en consola

Alcance: Implementado en Java 17+ sin dependencias (paquete com.libreriasmart). Se compila con compilar.bat (javac -encoding UTF8 -d out) y se ejecuta con run.bat (java -cp out com.libreriasmart.Main). El programa supera lo mínimo pedido: gestiona 12 productos, 5 clientes y ventas con descuentos, fidelización y reportes.

Prueba: Demostración mínima (2 objetos, 3 métodos): se vende 1 unidad del producto L-001 al cliente C-001. Objetos: la Venta V-1001 (creada) y el Cliente C-001 (existente). Métodos invocados: crearVenta(), calcularDescuento() —vía IPoliticaDescuento— y getTotalFinal(). Salida real capturada de la ejecución:

```
Catálogo: [1] L-001 · Cien años de soledad ($45.000, stock 15) ...
Cliente: Sofía Ramírez · Nivel DIAMANTE (descuento base 15%)
Agregado: Cien años de soledad x1
================ FACTURA V-1001 ================
Cliente : Sofía Ramírez (DIAMANTE)
ID     PRODUCTO              CANT  P.UNIT    SUBTOTAL
L-001  Cien años de soledad  1     $45.000   $45.000
Subtotal : $45.000
Descuento aplicado (15%): -$6.750
IVA 19%  : $7.268
TOTAL    : $45.518
Puntos de Sofía Ramírez ahora: 1204 (DIAMANTE)
```

```
Sugerencias: L-004, L-006, U-003
```
Lectura: La salida confirma el polimorfismo (se cobra un Producto sin saber su subtipo), la inyección de la política de descuento (15 % por nivel Diamante) y la fidelización (puntos 1200 → 1204).

8. Verificación de cumplimiento

Cruce entre lo pedido en la guía y su evidencia en este documento.

Tabla 3

Lista de chequeo de productos esperados.

Pedido	**Evidencia**	**Estado**
Diagrama refinado v1.0 (atributos, operaciones, interfaces, relaciones)	Figura 1 (§2) + Figura 2 Fase 1 (§1)	Cumple
Visibilidades (+, -, #), tipos y firmas	Diccionario, Tabla 1 (§3)	Cumple
≥1 interfaz + clases que la implementan	3 interfaces ITabulable, IInventariable, IPoliticaDescuento (§4)	Cumple
Relaciones con tipo, multiplicidad y justificación	Tabla 2, 13 relaciones (§5)	Cumple
Párrafo modelo del mundo + párrafo modelo de la interfaz	Sección 6	Cumple
Prototipo: ≥2 objetos, 2–3 métodos, salida en pantalla	Venta V-1001 + Cliente C-001; crearVenta, calcularDescuento, getTotalFinal (§7)	Cumple
Documento Word con normas APA	Portada, índice, figuras/tablas numeradas, referencias	Cumple

Referencias

Barnes, D. J. (2007). Programación orientada a objetos con Java: una introducción práctica usando BlueJ. Pearson Educación.

Deitel, H. M. y Deitel, P. J. (2016). Cómo programar en Java (10.ª ed.). Pearson.

Joyanes, L. (1996). Programación orientada a objetos. McGraw-Hill.

López, R. L. (2006). Metodología de la programación orientada a objetos. Alfaomega.

Wu, C. (2008). Programación en Java: introducción a la programación orientada a objetos. McGraw-Hill.